Pointers, Linked Lists, Stack และ Queue

ปรับจากเนื้อหา 01418113 โดย ชาคริต วัชโรภาส

ทำไมต้องเรียน Pointer?

นักเรียน TOI ส่วนใหญ่ใช้ **C++** และมี STL ที่จัดการ memory ให้อัตโนมัติแล้ว
แต่การเข้าใจ pointer ยังจำเป็นเพราะ

1. **เข้าใจว่า STL ทำงานยังไงข้างใน** — vector , list , map ล้วนใช้ pointer
2. **Debug ได้เร็วขึ้น** — เข้าใจว่า segfault เกิดจากอะไร
3. **Linked List ออกสอบ** — บางโจทย์ต้องสร้างโครงสร้างเอง
4. **Pass by reference** — เข้าใจความแตกต่างระหว่าง f(x) และ f(&x) และ f(*x)

> **เครื่องมือช่วย:** C++ Visualization (https://pythontutor.com/cpp.html) — ดูการทำงานของ pointer ในหน่วยความจำแบบ step-by-step

---

Part 1: Pointers

---

1. Pointer คืออะไร?

Pointer คือตัวแปรที่เก็บ **address** (ตำแหน่งในหน่วยความจำ) ของตัวแปรอื่น

```
หน่วยความจำ:

address:  1000  1004  1008  1012
value:    [ 5 ] [   ] [1000] [   ]
name:      c           p
```

- ตัวแปร c อยู่ที่ address 1000 มีค่า 5
- ตัวแปร p อยู่ที่ address 1008 มีค่า **1000** (เก็บ address ของ c)
- เรียกว่า p **ชี้ไปที่** c

ประกาศ Pointer ใน C++

In [ ]:

```cpp
#include <iostream>
using namespace std;

int main() {
    int c = 5;
    int *p;   // p คือ pointer ที่ชี้ไปที่ข้อมูลประเภท int

    p = &c;   // & = address operator: ดึง address ของ c

    cout << "c   = " << c   << "\n";  // 5
    cout << "*p  = " << *p  << "\n";  // 5  (*p = dereference = ดูค่าที่ p ชี้อยู่)
    cout << "&c  = " << &c  << "\n";  // address ของ c
    cout << "p   = " << p   << "\n";  // address เดียวกัน
    cout << "&p  = " << &p  << "\n";  // address ของ p เอง (ต่างออกไป)

    return 0;
}
```

สรุป Operators

Operator	ชื่อ	ความหมาย	ตัวอย่าง
&	address-of	ดึง address ของตัวแปร	p = &c
*	dereference	เข้าถึงค่าที่ pointer ชี้อยู่	*p = 10
->	member access	เข้าถึง field ของ struct ผ่าน pointer	p->data

`p->data` เป็น shorthand ของ `(*p).data` — dereference แล้วเข้าถึง field ในบรรทัดเดียว
จะใช้บ่อยมากเมื่อสร้าง linked list ใน Part 2

> **จำง่าย:** `&` = "address ของ", `*` = "ค่าที่ address นั้น", `->` = "field ของ struct ที่ pointer ชี้อยู่"

In [ ]:

```cpp
#include <iostream>
using namespace std;

struct Point {
    int x, y;
};

int main() {
    Point p = {3, 4};
    Point *ptr = &p;

    // สองแบบนี้ได้ผลเหมือนกัน
    cout << (*ptr).x << "\n";  // 3 — dereference แล้วเข้าถึง field
    cout << ptr->x   << "\n";  // 3 — shorthand ที่นิยมใช้

    ptr->x = 10;  // แก้ค่า x ผ่าน pointer
    cout << p.x << "\n";  // 10 — p เปลี่ยนด้วย

    return 0;
}
```

🔧 แบบฝึกหัด 1.1 — ติดตามค่า

โค้ดนี้จะพิมพ์อะไร? ลองคิดก่อนรัน

In [ ]:

```cpp
#include <iostream>
using namespace std;

int main() {
    int x = 1, y = 2;
    int *p;

    p  = &x;   // p ชี้ไปที่ x
    y  = *p;   // y = ?
    *p = 0;    // x = ?
    p  = &y;   // p ย้ายไปชี้ที่ y

    cout << "x = " << x << ", y = " << y << "\n";
    // คำตอบคือ: x = ?, y = ?
    return 0;
}
```

---

2. Pass by Reference — ส่ง address โดยไม่ต้องใช้ *

ใน C ถ้าต้องการให้ฟังก์ชันแก้ค่าตัวแปรภายนอก ต้องส่ง address และใช้ `*` ตลอด

```cpp
// C-style
void double_it(int *p) { *p = *p * 2; }
double_it(&a);  // ต้องส่ง &a
```

C++ มี **reference** (`&`) ที่ทำสิ่งเดียวกัน แต่ syntax สะอาดกว่ามาก — ไม่ต้องใช้ `*` และไม่ต้องส่ง `&a`

In [ ]:

```cpp
#include <iostream>
using namespace std;

void double_value(int &x) {  // & = reference — แก้ตัวจริงได้เลย
    x = x * 2;
}

void swap_values(int &a, int &b) {
    int tmp = a;
    a = b;
    b = tmp;
}

int main() {
    int a = 5, b = 10;

    double_value(a);
    cout << "a = " << a << "\n";  // 10

    swap_values(a, b);
    cout << "a = " << a << ", b = " << b << "\n";  // a = 10, b = 10?
    // ลองคิดดูว่า output จะเป็นอะไร — a ถูก double แล้วก่อน swap
    return 0;
}
```

เมื่อไหร่ใช้อะไร

แบบ	Syntax	ใช้เมื่อ
Pass by value	f(int x)	ข้อมูลเล็ก ไม่ต้องแก้ตัวจริง
Pass by pointer	f(int *p)	Linked list — Section 7
Pass by reference	f(int &x)	**แนะนำใน C++** — swap, แก้ค่า, ส่ง container ใหญ่

จำง่าย: ใน C++ ใช้ & เป็น default — เปลี่ยนไปใช้ * เฉพาะเมื่อสร้าง linked list หรือ dynamic structure

---

3. Pointer กับ Array

ชื่อ array ใน C/C++ **คือ pointer ไปยัง element แรก**

In [ ]:

```cpp
#include <iostream>
using namespace std;

int main() {
    int a[] = {10, 20, 30, 40, 50};

    // a และ &a[0] คือสิ่งเดียวกัน
    cout << a     << "\n";   // address ของ a[0]
    cout << &a[0] << "\n";   // เหมือนกัน

    // a[i] และ *(a+i) คือสิ่งเดียวกัน
    cout << a[2]    << "\n";  // 30
    cout << *(a+2)  << "\n";  // 30 — เลื่อน pointer 2 ตัว

    // pointer arithmetic
    int *p = a;       // p ชี้ที่ a[0]
    p++;              // p ชี้ที่ a[1] (เลื่อนไป sizeof(int) = 4 bytes)
    cout << *p << "\n";  // 20

    return 0;
}
```

⚠️ Array Decay — จุดที่มักงง

เมื่อส่ง array เข้าฟังก์ชัน มันกลายเป็น pointer — **ไม่รู้ขนาดแล้ว**

In [ ]:
```cpp
#include <iostream>
using namespace std;

void print_size(int arr[]) {
    // arr ที่นี่คือ int* ไม่ใช่ array จริงๆ
    cout << "ในฟังก์ชัน: sizeof(arr) = " << sizeof(arr) << "\n";  // 8 (ขนาด pointer)
}

int main() {
    int buf[10] = {1,2,3,4,5,6,7,8,9,10};
    cout << "ใน main:     sizeof(buf) = " << sizeof(buf) << "\n";  // 40 (10 * 4)
    print_size(buf);
    return 0;
}
// วิธีแก้: ส่ง size ไปด้วย หรือใช้ vector<int> แทน
```

> **บทเรียน:** ในการแข่งขัน ใช้ `vector<int>` แทน raw array เสมอ — ไม่มีปัญหา decay และส่งเข้าฟังก์ชันได้สะดวก

---

4. Dynamic Memory Allocation

หน่วยความจำ 3 ส่วน

```
┌──────────────┐
│    Stack     │  ← ตัวแปร local, function calls (จัดการอัตโนมัติ)
├──────────────┤
│     ...      │
├──────────────┤
│    Heap      │  ← malloc/new (เราจัดการเอง)
├──────────────┤
│    Data      │  ← ตัวแปร global, static
│    Code      │  ← โค้ดโปรแกรม
└──────────────┘
```

ใน C++ ใช้ `new` และ `delete` (ไม่ใช่ malloc/free)

In [ ]:
```cpp
#include <iostream>
using namespace std;

int main() {
    // จอง int 1 ตัวบน Heap
    int *p = new int;    // จอง
    *p = 42;
    cout << *p << "\n";  // 42
    delete p;            // คืน

    // จอง array บน Heap
    int n = 5;
    int *arr = new int[n];  // จอง array ขนาด n
    for (int i = 0; i < n; i++) arr[i] = i * 10;
    for (int i = 0; i < n; i++) cout << arr[i] << " ";
    cout << "\n";
    delete[] arr;           // คืน array (ใช้ [] ด้วย)

    return 0;
}
```

⚠️ Segmentation Fault

เกิดเมื่อ **เข้าถึงหน่วยความจำที่ไม่ได้รับอนุญาต**
สาเหตุที่พบบ่อยในการแข่งขัน:

```
int *p;          // ประกาศแต่ไม่กำหนดค่า — p ชี้ที่ไหนก็ไม่รู้
*p = 5;          // SEGFAULT!

int a[5];
a[10] = 1;       // out of bounds — SEGFAULT!

int *q = nullptr;
*q = 5;          // dereference null pointer — SEGFAULT!

Node *tmp = head->next;  // ถ้า head เป็น NULL — SEGFAULT!
```

> **กฎสำคัญ:** ก่อน dereference pointer เสมอ ให้เช็คว่าไม่เป็น `nullptr` ก่อน

Part 2: Linked Lists

5. Self-Referential Struct — รากฐานของ Linked List

Node คือ struct ที่มี pointer ชี้กลับมาหาตัวเอง

In [ ]:

```
#include <iostream>
using namespace std;

struct Node {
    int data;       // ข้อมูล
    Node *next;     // pointer ไปยัง Node ถัดไป
};

int main() {
    Node a, b, c;

    a.data = 10; a.next = &b;
    b.data = 20; b.next = &c;
    c.data = 30; c.next = nullptr;  // ตัวสุดท้าย → NULL

    // traverse
    for (Node *tmp = &a; tmp != nullptr; tmp = tmp->next)
        cout << tmp->data << " ";  // 10 20 30
    cout << "\n";

    return 0;
}
```

ภาพ Linked List

```
head
 |
 ▼
[10 | •]──→[20 | •]──→[30 | NULL]
```

- `head` ชี้ไปที่ node แรก
- `->` เป็น shorthand ของ `(*ptr).field`
- Node สุดท้าย `next = nullptr`

---

6. การสร้าง Linked List ด้วย Dynamic Allocation

6.1 สร้างด้วย Stack Style (Prepend — ใส่หน้า)

In [ ]:

```cpp
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node *next;
};

int main() {
    Node *head = nullptr;

    for (int val : {1, 2, 3}) {
        Node *new_node = new Node{val, head};
        head = new_node;
    }

    for (Node *tmp = head; tmp; tmp = tmp->next)
        cout << tmp->data << " ";
    cout << "\n";
    return 0;
}
```

6.2 สร้างด้วย Queue Style (Append — ใส่หลัง)

ถ้าต้องการลำดับเดียวกับ input ต้องใช้ pointer tail เพิ่ม

In [ ]:

```cpp
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node *next;
};

int main() {
    Node *head = nullptr, *tail = nullptr;

    for (int val : {1, 2, 3, 4, 5}) {
        Node *new_node = new Node{val, nullptr};
        if (head == nullptr) {
            head = tail = new_node;
        } else {
            tail->next = new_node;
            tail = new_node;
        }
    }

    for (Node *tmp = head; tmp; tmp = tmp->next)
        cout << tmp->data << " ";
    cout << "\n";

    return 0;
}
```

🔧 แบบฝึกหัด 6.1

รับตัวเลข N ตัวจาก input สร้าง linked list แบบ queue style แล้วพิมพ์ออกมา

In [ ]:

```cpp
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node *next;
};

int main() {
    int n;
    cin >> n;

    Node *head = nullptr, *tail = nullptr;

    for (int i = 0; i < n; i++) {
        int val;
        cin >> val;
        // TODO: สร้าง node และเพิ่มเข้า list
    }

    // TODO: print list

    return 0;
}
```

---

7. Double Pointer — `Node **head`

ปัญหา: แก้ head จากในฟังก์ชันไม่ได้

ถ้าส่ง `Node *head` เข้าฟังก์ชัน การแก้ `head = ...` ภายในฟังก์ชันจะไม่มีผลภายนอก

In [ ]:

```cpp
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node *next;
};

// แบบผิด — head ภายนอกไม่เปลี่ยน
void push_wrong(Node *head, int val) {
    Node *n = new Node;
    n->data = val;
    n->next = head;
    head = n;  // แก้แค่ copy ของ pointer ในฟังก์ชัน!
}

// แบบถูก — ส่ง pointer ของ pointer
void push(Node **head, int val) {
    Node *n = new Node;
    n->data = val;
    n->next = *head;  // *head = ค่าของ head เดิม
    *head = n;        // แก้ head ตัวจริง
}

int main() {
    Node *head = nullptr;

    push_wrong(head, 10);
    cout << "After push_wrong: " << (head ? head->data : -1) << "\n";  // -1 (head ยังเป็น null)

    push(&head, 10);
    push(&head, 20);
    push(&head, 30);
    for (Node *tmp = head; tmp; tmp = tmp->next)
        cout << tmp->data << " ";  // 30 20 10
    cout << "\n";

    return 0;
}
```

ทำความเข้าใจ **head

```
head (Node*)     → ชี้ไปที่ node แรก
&head (Node**)   → address ของตัวแปร head เอง

ในฟังก์ชัน:
void push(Node **head, ...)  — head ตรงนี้คือ address ของ pointer ภายนอก
*head                        — คือค่าของ pointer ภายนอก (ตัว node แรก)
**head                       — คือ node แรก
```

> **ทางเลือกใน C++:** ใช้ reference แทน double pointer ได้
> `void push(Node *&head, int val)` — อ่านง่ายกว่า

In [ ]:

```cpp
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node *next;
};

// C++ style: ใช้ reference to pointer
void push(Node *&head, int val) {
    Node *n = new Node;
    n->data = val;
    n->next = head;
    head = n;  // แก้ head ตัวจริงผ่าน reference
}

int main() {
    Node *head = nullptr;
    push(head, 10);  // ไม่ต้องส่ง &head
    push(head, 20);
    push(head, 30);
    for (Node *tmp = head; tmp; tmp = tmp->next)
        cout << tmp->data << " ";  // 30 20 10
    cout << "\n";
    return 0;
}
```

---

8. Linked List Operations

8.1 Traversal (วนลูป)

In [ ]:

```cpp
#include <iostream>
using namespace std;

struct Node { int data; Node *next; };

void print_list(Node *head) {
    for (Node *tmp = head; tmp != nullptr; tmp = tmp->next)
        cout << tmp->data << " → ";
    cout << "NULL\n";
}

int length(Node *head) {
    int cnt = 0;
    for (Node *tmp = head; tmp; tmp = tmp->next)
        cnt++;
    return cnt;
}
```

8.2 Insertion

In [ ]:

```
#include <iostream>
using namespace std;

struct Node { int data; Node *next; };

// ใส่หน้า list
void insert_front(Node *&head, int val) {
    Node *n = new Node{val, head};
    head = n;
}

// ใส่หลัง node ที่กำหนด
void insert_after(Node *prev, int val) {
    if (!prev) return;
    Node *n = new Node{val, prev->next};
    prev->next = n;
}

void print_list(Node *head) {
    for (Node *tmp = head; tmp; tmp = tmp->next)
        cout << tmp->data << " ";
    cout << "\n";
}

int main() {
    Node *head = nullptr;
    insert_front(head, 30);  // 30
    insert_front(head, 20);  // 20 → 30
    insert_front(head, 10);  // 10 → 20 → 30
    print_list(head);

    insert_after(head, 15);  // 10 → 15 → 20 → 30
    print_list(head);

    return 0;
}
```

8.3 Deletion

In [ ]:

```cpp
#include <iostream>
using namespace std;

struct Node { int data; Node *next; };

// ลบ node ที่มี data == target
void delete_node(Node *&head, int target) {
    // กรณี 1: ลบ head
    if (head && head->data == target) {
        Node *tmp = head;
        head = head->next;
        delete tmp;
        return;
    }
    // กรณี 2: ลบตัวกลาง/ท้าย
    for (Node *tmp = head; tmp && tmp->next; tmp = tmp->next) {
        if (tmp->next->data == target) {
            Node *to_del = tmp->next;
            tmp->next = to_del->next;
            delete to_del;
            return;
        }
    }
    // กรณี 3: ไม่เจอ — ไม่ทำอะไร
}

void print_list(Node *head) {
    for (Node *tmp = head; tmp; tmp = tmp->next)
        cout << tmp->data << " ";
    cout << "\n";
}

int main() {
    // สร้าง: 10 → 20 → 30 → 40
    Node *head = nullptr;
    for (int v : {40, 30, 20, 10}) {
        Node *n = new Node{v, head};
        head = n;
    }
    print_list(head);          // 10 20 30 40

    delete_node(head, 10);     // ลบ head
    print_list(head);          // 20 30 40

    delete_node(head, 30);     // ลบกลาง
    print_list(head);          // 20 40

    delete_node(head, 99);     // ไม่เจอ — ไม่เปลี่ยนอะไร
    print_list(head);          // 20 40

    return 0;
}
```

🔧 แบบฝึกหัด 8.1 — Reverse Linked List

เขียนฟังก์ชัน `reverse(Node *&head)` ที่กลับลำดับ list
เช่น `1 → 2 → 3 → 4` กลายเป็น `4 → 3 → 2 → 1`
Hint: ใช้ 3 pointer: `prev`, `curr`, `next`

In [ ]:

```cpp
#include <iostream>
using namespace std;

struct Node { int data; Node *next; };

void reverse(Node *&head) {
    Node *prev = nullptr;
    Node *curr = head;
    // TODO: implement
}

void print_list(Node *head) {
    for (Node *tmp = head; tmp; tmp = tmp->next)
        cout << tmp->data << " ";
    cout << "\n";
}

int main() {
    Node *head = nullptr;
    for (int v : {4, 3, 2, 1}) {
        Node *n = new Node{v, head};
        head = n;
    }
    print_list(head);   // 1 2 3 4
    reverse(head);
    print_list(head);   // 4 3 2 1
    return 0;
}
```

---

9. Stack บน Singly Linked List

Stack คือโครงสร้างแบบ **LIFO (Last In First Out)** — ใส่ทีหลังออกก่อน
ใช้ singly linked list โดย push และ pop ที่ **head** เสมอ — O(1) ทั้งคู่

```
push(10): [10] → NULL
push(20): [20] → [10] → NULL
push(30): [30] → [20] → [10] → NULL
pop():    [20] → [10] → NULL    ← คืน 30
top():    20                    ← แค่ดู ไม่ลบ
```

ฟังก์ชัน	ความหมาย	ชื่อใน STL stack
push(x)	ใส่ที่ head	push(x)
pop()	ลบที่ head	pop()
top()	ดูค่าที่ head	top()
empty()	เช็คว่าว่าง	empty()
size()	จำนวน element	size()

In [ ]:

```cpp
#include <bits/stdc++.h>
using namespace std;

struct Node {
    int data;
    Node *next;
};

struct Stack {
    Node *head = nullptr;
    int  sz    = 0;

    bool empty() { return head == nullptr; }
    int  size()  { return sz; }

    int top() {
        return head->data;  // ห้ามเรียกเมื่อ empty
    }

    void push(int val) {
        Node *n = new Node{val, head};
        head = n;
        sz++;
    }

    void pop() {
        if (empty()) return;
        Node *tmp = head;
        head = head->next;
        delete tmp;
        sz--;
    }
};

int main() {
    Stack s;
    s.push(10);
    s.push(20);
    s.push(30);

    cout << "size  = " << s.size()  << "\n";  // 3
    cout << "top() = " << s.top()   << "\n";  // 30

    while (!s.empty()) {
        cout << s.top() << " ";  // 30 20 10
        s.pop();
    }
    cout << "\n";
    return 0;
}
```

10. Queue บน Singly Linked List

Queue คือโครงสร้างแบบ **FIFO (First In First Out)** — ใส่ทีแรกออกก่อน
ใช้ singly linked list โดย push ที่ **tail** และ pop ที่ **head** — O(1) ทั้งคู่

```
push(10): head→[10]←tail
push(20): head→[10]→[20]←tail
push(30): head→[10]→[20]→[30]←tail
pop():    head→[20]→[30]←tail   ← คืน 10
front():  20                    ← แค่ดู ไม่ลบ
back():   30
```

ฟังก์ชัน	ความหมาย	ชื่อใน STL queue
push(x)	ใส่ที่ tail	push(x)

ฟังก์ชัน	ความหมาย	ชื่อใน STL queue
pop()	ลบที่ head	pop()
front()	ดูค่าที่ head	front()
back()	ดูค่าที่ tail	back()
empty()	เช็คว่าว่าง	empty()

In [ ]:

```cpp
#include <bits/stdc++.h>
using namespace std;

struct Node {
    int data;
    Node *next;
};

struct Queue {
    Node *head = nullptr;
    Node *tail = nullptr;
    int   sz   = 0;

    bool empty() { return head == nullptr; }
    int  size()  { return sz; }

    int front() { return head->data; }  // ห้ามเรียกเมื่อ empty
    int back()  { return tail->data; }  // ห้ามเรียกเมื่อ empty

    void push(int val) {
        Node *n = new Node{val, nullptr};
        if (empty()) head = tail = n;  // list ว่าง
        else { tail->next = n; tail = n; }
        sz++;
    }

    void pop() {
        if (empty()) return;
        Node *tmp = head;
        head = head->next;
        if (!head) tail = nullptr;  // list ว่างแล้ว
        delete tmp;
        sz--;
    }
};

int main() {
    Queue q;
    q.push(10);
    q.push(20);
    q.push(30);

    cout << "size   = " << q.size()  << "\n";  // 3
    cout << "front  = " << q.front() << "\n";  // 10
    cout << "back   = " << q.back()  << "\n";  // 30

    while (!q.empty()) {
        cout << q.front() << " ";  // 10 20 30
        q.pop();
    }
    cout << "\n";
    return 0;
}
```

11. Doubly Linked List

แต่ละ node มี pointer ไปทั้ง **ถัดไป** และ **ก่อนหน้า**

```
NULL ←[10]⇆[20]⇆[30]→ NULL
```

In [ ]:

```cpp
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node *prev, *next;
};

void insert_front(Node *&head, int val) {
    Node *n = new Node{val, nullptr, head};
    if (head) head->prev = n;  // ถ้ามี head เดิม ให้ head->prev ชี้กลับมา
    head = n;
}

int main() {
    Node *head = nullptr;
    insert_front(head, 30);
    insert_front(head, 20);
    insert_front(head, 10);

    // เดินหน้า
    cout << "Forward:  ";
    for (Node *tmp = head; tmp; tmp = tmp->next)
        cout << tmp->data << " ";  // 10 20 30
    cout << "\n";

    // หา tail
    Node *tail = head;
    while (tail->next) tail = tail->next;

    // เดินถอยหลัง
    cout << "Backward: ";
    for (Node *tmp = tail; tmp; tmp = tmp->prev)
        cout << tmp->data << " ";  // 30 20 10
    cout << "\n";

    return 0;
}
```

---

12. Circular Linked List

Circular linked list คือ linked list ที่ node สุดท้าย next **ชี้กลับมาที่ head** แทน nullptr

```
singly:   [10] → [20] → [30] → NULL
circular: [10] → [20] → [30] ─┐
           ↑                  │
           └──────────────────┘
```

ผลที่ตามมา:

- traverse ต้องเปลี่ยนเงื่อนไขหยุดจาก tmp != nullptr เป็น tmp != head
- เดินจาก node ไหนก็ไปถึงทุก node ได้ — ไม่มีจุดสิ้นสุด
- ใช้งานในสถานการณ์ที่ต้องวนซ้ำ เช่น round-robin scheduling

In [ ]:

```cpp
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node *next;
};

void append(Node *&head, int val) {
    Node *n = new Node{val, nullptr};
    if (!head) {
        head = n;
        n->next = head;
    } else {
        Node *tail = head;
        while (tail->next != head) tail = tail->next;
        tail->next = n;
        n->next = head;
    }
}

void print_circular(Node *head) {
    if (!head) return;
    Node *tmp = head;
    do {
        cout << tmp->data << " ";
        tmp = tmp->next;
    } while (tmp != head);  // หยุดเมื่อวนกลับมาที่ head
    cout << "\n";
}

int main() {
    Node *head = nullptr;
    append(head, 10);
    append(head, 20);
    append(head, 30);

    print_circular(head);  // 10 20 30
    return 0;
}
```

---

13. Circular Linked List with Dummy Node

ทำไมต้องใช้?

การ insert/delete บน doubly linked list มี **edge case** หลายอย่าง:

- ลบ head → ต้องแก้ head
- ลบ tail → ต้องเช็ค nullptr
- list ว่าง → เช็คพิเศษ

นำ circular มารวมกับ doubly และเพิ่ม **sentinel (dummy) node** ที่ไม่เก็บข้อมูล เพื่อแก้ปัญหานี้ทั้งหมด — ไม่มี edge case เลย!

In [ ]:

```cpp
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node *prev, *next;
};

// โครงสร้าง:
// dummy.next → node แรก, dummy.prev → node สุดท้าย
// list ว่าง: dummy.next == dummy.prev == &dummy

// ใส่ node n ก่อน node 'before'
void insert_before(Node *before, Node *n) {
    n->next = before;
    n->prev = before->prev;
    before->prev->next = n;
    before->prev = n;
}

// ลบ node n ออกจาก list
void remove_node(Node *n) {
    n->prev->next = n->next;
    n->next->prev = n->prev;
    delete n;
}

int main() {
    Node dummy;
    dummy.prev = dummy.next = &dummy;  // circular: ชี้หาตัวเอง
    Node *head = &dummy;

    // ใส่ที่ท้าย = insert_before(dummy) เพราะ dummy = ตัวสุดท้าย+1
    for (int v : {10, 20, 30, 40, 50}) {
        Node *n = new Node{v, nullptr, nullptr};
        insert_before(head, n);  // ใส่ก่อน head = ใส่ท้าย
    }

    // print forward
    cout << "Forward:  ";
    for (Node *tmp = head->next; tmp != head; tmp = tmp->next)
        cout << tmp->data << " ";
    cout << "\n";

    // print backward
    cout << "Backward: ";
    for (Node *tmp = head->prev; tmp != head; tmp = tmp->prev)
        cout << tmp->data << " ";
    cout << "\n";

    // ลบ node ที่มีค่า 30
    for (Node *tmp = head->next; tmp != head; tmp = tmp->next) {
        if (tmp->data == 30) {
            remove_node(tmp);
            break;
        }
    }

    cout << "After delete 30: ";
    for (Node *tmp = head->next; tmp != head; tmp = tmp->next)
        cout << tmp->data << " ";
    cout << "\n";

    return 0;
}
```

ทำไม insert_before ไม่ต้องเช็ค edge case?

```
ก่อน insert:
[dummy] ⇆ [10] ⇆ [20] ⇆ [dummy]  (circular)

insert_before(dummy, node{30}):
[dummy] ⇆ [10] ⇆ [20] ⇆ [30] ⇆ [dummy]

insert_before(dummy, node{10}) เมื่อ list ว่าง:
[dummy] ⇆ [10] ⇆ [dummy]  — โค้ดเดียวกัน ไม่มี if!
```

นี่คือ pattern ที่ใช้ภายใน **`std::list`** ของ C++ STL

---

14. Linked List vs Vector — เลือกอะไร?

	Linked List	vector / deque
Insert/Delete ที่ตำแหน่งรู้แล้ว	**O(1)**	O(N)
ค้นหาตำแหน่ง	O(N)	O(N) / O(1)
Random access a[i]	O(N)	**O(1)**
Cache friendly?	❌	✅
ใช้ใน contest	น้อย	**บ่อยมาก**

เมื่อไหร่ใช้ Linked List ใน contest?

- โจทย์บังคับให้ implement linked list
- ต้องการ insert/delete ตรงกลางบ่อยมาก โดยรู้ตำแหน่งอยู่แล้ว
- `std::list<T>` ใน STL (ใช้ได้เลย ไม่ต้อง implement เอง)

> **ในทางปฏิบัติ:** ส่วนใหญ่ใช้ vector + deque + set แก้ได้เกือบทุกโจทย์
> Linked List เป็นความรู้พื้นฐานที่ช่วยให้เข้าใจ STL ข้างใน

Operation	Linked List (เขียนเอง)	std::list<T>
ใส่หน้า	เขียน push_front เอง	l.push_front(x)
ใส่หลัง	เขียน push_back เอง	l.push_back(x)
ใส่กลาง	หา iterator แล้วเขียนเอง	l.insert(it, x)
ลบหน้า	เขียน pop_front เอง	l.pop_front()
ลบหลัง	เขียน pop_back เอง	l.pop_back()
ลบกลาง	แก้ pointer เอง	l.erase(it)
วนลูป	for(tmp=head; tmp; tmp=tmp->next)	for(auto x : l)
จำนวน element	นับเอง	l.size()
กลับลำดับ	เขียน algorithm เอง	l.reverse()
เรียงลำดับ	เขียน merge sort เอง	l.sort()
รวม 2 list ที่เรียงแล้ว	เขียนเอง	l.merge(l2)

> ⚠️ **ข้อควรระวังใน contest**
> `std::list` ไม่มี random access — `l[i]` ใช้ไม่ได้ และการหา iterator ด้วย `find()` ยังคงเป็น O(N) อยู่ดี ดังนั้น insert/delete กลางจะได้ O(1) **ก็ต่อเมื่อมี iterator อยู่แล้วเท่านั้น**
> ในทางปฏิบัติ vector + index ยังเป็นตัวเลือกที่นิยมกว่า `std::list` เพราะ cache friendly กว่ามาก และโจทย์ส่วนใหญ่ไม่ได้ต้องการ O(1) insert กลางจริงๆ

In [ ]:

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    list<int> l = {1, 2, 4, 5};

    // หา iterator ชี้ที่ 4
    auto it = find(l.begin(), l.end(), 4);

    // insert 3 ก่อน 4 — O(1)
    l.insert(it, 3);

    // ตอนนี้: 1 2 3 4 5
    for (int x : l) cout << x << " ";
    cout << "\n";

    // erase 3 — O(1)
    auto it2 = find(l.begin(), l.end(), 3);
    l.erase(it2);

    // ตอนนี้: 1 2 4 5
    for (int x : l) cout << x << " ";
    cout << "\n";

    return 0;
}
```

15. สรุป

Pointer

แนวคิด	สิ่งที่ต้องจำ
&x	address ของ x
*p	ค่าที่ p ชี้อยู่
p->field	shorthand ของ (*p).field
pass by *	C-style, ต้องส่ง &x
pass by &	C++-style, นิยมกว่า
new / delete	จอง/คืน heap
nullptr check	ก่อน dereference เสมอ

Linked List

แนวคิด	สิ่งที่ต้องจำ
Prepend	new_node->next = head; head = new_node
Append	ต้องมี tail pointer
Node ** / Node *&	ต้องการเมื่อแก้ head จากฟังก์ชัน
Deletion	เช็ค 3 กรณี: head / กลาง / ไม่เจอ
Doubly	มี prev และ next
Circular	node สุดท้าย next ชี้กลับมาที่ head — traverse ใช้ do...while
Circular + dummy	ไม่มี edge case — ใช้ใน std::list

Stack และ Queue บน Linked List

	Stack	Queue
หลักการ	LIFO	FIFO
push	ใส่ที่ head	ใส่ที่ tail
pop	ลบที่ head	ลบที่ head
ดูค่าหน้า	top()	front()
ดูค่าหลัง	—	back()
Complexity	O(1) ทุก operation	O(1) ทุก operation

🏋️ แบบฝึกหัดเพิ่มเติม

ข้อ 1: เขียนฟังก์ชัน `find_middle(Node *head)` ที่หา node ที่อยู่ตรงกลางของ list โดยผ่านแค่ครั้งเดียว (one pass)
Hint: ใช้ two pointer — slow เดินทีละ 1, fast เดินทีละ 2

ข้อ 2: เขียนฟังก์ชัน `has_cycle(Node *head)` ที่ตรวจว่า linked list มี cycle หรือไม่
Hint: Floyd's algorithm — slow/fast pointer

ข้อ 3: Merge two sorted linked lists ให้ได้ list ที่ยังเรียงอยู่

ข้อ 4: เขียน Stack ที่มี `min()` คืนค่าต่ำสุดใน stack ได้ใน O(1)
Hint: ใช้ stack สองอัน อันหนึ่งเก็บข้อมูล อีกอันเก็บค่า min ณ แต่ละ level

ข้อ 5: เขียน Queue โดยใช้ Stack สองอัน (ห้ามใช้ linked list โดยตรง) **Hint:**

- `inbox` — รับข้อมูลเข้าเสมอ
- `outbox` — ส่งข้อมูลออกเสมอ

```
push(x):
    inbox.push(x)

pop():
    ถ้า outbox ว่าง:
        ย้ายทุกตัวจาก inbox → outbox
    outbox.pop()

front():
    ถ้า outbox ว่าง:
        ย้ายทุกตัวจาก inbox → outbox  (เหมือน pop)
    คืน outbox.top()

empty():
    คืน inbox.empty() AND outbox.empty()
```

In [ ]:

```cpp
// ข้อ 3: Merge two sorted linked lists ให้ได้ list ที่ยังเรียงอยู่
#include <bits/stdc++.h>
using namespace std;

struct Node {
    int data;
    Node *next;
    Node(int d) : data(d), next(nullptr) {}
};

// สร้าง linked list จาก array
Node* make_list(vector<int> vals) {
    Node *head = nullptr, *tail = nullptr;
    for (int v : vals) {
        Node *n = new Node(v);
        if (!head) head = tail = n;
        else { tail->next = n; tail = n; }
    }
    return head;
}

// พิมพ์ linked list
void print_list(Node *head) {
    for (Node *p = head; p; p = p->next)
        cout << p->data << (p->next ? " -> " : "\n");
}

// TODO: เขียนฟังก์ชันนี้
/*    merge(a, b):
          ถ้า a หมด → คืน b
          ถ้า b หมด → คืน a

          ถ้า a.data ≤ b.data:
              a.next = merge(a.next, b)
              คืน a
          ไม่งั้น:
              b.next = merge(a, b.next)
              คืน b    */
Node* merge(Node *a, Node *b) {
    // ...
}

int main() {
    Node *a = make_list({1, 3, 5, 7, 9});
    Node *b = make_list({2, 4, 6, 8, 10});

    cout << "list a: "; print_list(a);
    cout << "list b: "; print_list(b);

    Node *merged = merge(a, b);

    cout << "merged: "; print_list(merged);
    // expected: 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7 -> 8 -> 9 -> 10

    return 0;
}
```